

# STATE LEARNING CHEAT SHEET #2

## Anatomy of an LLM Application

Frontend, backend, database, model API, application logic, context construction, structured outputs, and validation.

### The big picture

An LLM is one component inside an application. The surrounding software controls what the model sees, what it may influence, what is persisted, and what happens when it fails.

| Component         | Job                                                             | State example                                          |
|-------------------|-----------------------------------------------------------------|--------------------------------------------------------|
| Frontend          | User-facing interface.                                          | Add Evidence, Review, Current State, Ask.              |
| Backend           | Server-side umbrella coordinating requests and protected logic. | Receives Evidence, calls model, reads/writes database. |
| Database          | Persists information across sessions.                           | Evidence, Current State, History, Reviews, Rules.      |
| LLM API           | Probabilistic language interpretation.                          | Interprets implications of new Evidence.               |
| Application logic | Enforces product rules and allowed transitions.                 | Prevents silent consequential State changes.           |
| Human             | Provides authority where approval is required.                  | Approves/rejects consequential transitions.            |

### Backend is an umbrella term

A backend can encompass request handling, database access, LLM calls, context construction, validation, application/business logic, and server-side security. These are separate **responsibilities**; they do not need to be separate servers.

**Logical separation != physical separation. Tiny State v1 can keep these responsibilities in one codebase.**

### One Evidence flow through State

- Frontend sends new Evidence to the backend.
- Backend saves the untouched source Evidence first.
- Backend constructs the context the model needs.
- LLM returns a structured interpretation/recommendation.
- Backend validates that response.
- Application logic creates Review if required; the model does not directly change State.
- Frontend shows source Evidence + AI interpretation + proposed change.
- Human decides; application performs an approved transition and preserves prior State in History.

### Context construction

**Context construction** means deciding what information the model receives for a particular task. It is part of product/application logic, not merely prompt writing.

State's simple starting package: **new Evidence + all small Current State + Project Rules + open Review issues**. Do not send the entire Evidence archive or History by default.

**Send the smallest trustworthy governing context sufficient for the task. Do not optimize token count so aggressively that important context is missed.**

### Structured outputs

Instead of unrestricted prose, State asks the model to fill a predefined response shape: topic, interpretation, consequential, why consequential, affected State items, Review needed, Review type, and proposed State change.

**The model is effectively filling out a recommendation form. It is not issuing database commands.**

## Validation

The backend checks whether model output is safe and usable before acting: required fields are present, values are allowed, referenced records exist, and required combinations make sense.

**Valid output != correct output. Validation can show that a response is well-formed enough for software to process; it cannot prove the model interpreted the Evidence correctly.**

## Fail safely

If model processing fails or returns unusable output, preserve the user's original Evidence but do not perform a consequential State change. State can report that interpretation failed and allow retry/recovery.

## Designing for model mistakes

State assumes the model will sometimes be wrong. Reliability comes partly from limiting and recovering from those errors:

- preserve original Evidence unchanged;
- keep AI interpretation separate from source material;
- validate output before application actions;
- require human authority for consequential transitions;
- show source Evidence beside AI interpretation in Review;
- preserve superseded Current State in History.

## Internal model vs. interaction model

The database can represent richer concepts than the UI asks the user to maintain. State may internally track Evidence, interpretations, Review relationships, resolutions, and provenance while keeping Review actions simple.

**Rich internal representation does not require a burdensome interface.**

## Current State and Ask

For ordinary current-state questions, Current State should govern without repeatedly re-reasoning over underlying Evidence. If a **relevant unresolved Review issue** could make it stale, Ask should surface that as a qualification rather than silently changing the answer.

## What you should be able to explain

- Why the LLM is only one part of an AI application.
- What frontend, backend, database, model API, and application logic each do.
- Why logical components do not require separate infrastructure.
- What context construction, structured output, and validation mean.
- Why valid model output can still be wrong.
- How surrounding software limits damage from model mistakes.

## One-sentence summary

**The model interprets ambiguity; the surrounding application controls context, persistence, validation, permissions, and consequences.**